01 / The project in plain language

This guide explains the **SomaiyaSat Mission Data Dashboard** as it is currently implemented. Read it alongside the code and practise the short demonstration on page 9.

A 30-second introduction
Our project is a desktop dashboard for organizing and viewing sample satellite mission data. We use Python for the program logic, Tkinter for the interface, and SQLite for local storage. Users can view satellite health, filter communication records, and inspect deployment events. All mission data is synthetic, so the application works offline.

What problem does it address?

Mission information is easier to understand when health readings, communication records, and deployment events are available in one place. The dashboard turns stored rows into tables, summary cards, and a battery chart.

Term	Meaning in this project
SomaiyaSat / SomaiyaPod	The satellite / the deployer that releases it.
Telemetry	Recorded health readings such as battery level and temperature.
Frontend	The window, buttons, filters, tables, and chart the user sees.
Backend	Python functions that validate input, query SQLite, and calculate results.
Database	A persistent collection of rows stored in data/mission.db.
Synthetic data	Invented, repeatable sample records for demonstrating the application.
TT&C / SSTV	Health and command data / image-transmission categories in the mission.
Codec2 / M17	Voice coding / digital radio communication. Here they are labels on sample records, not implemented radio services.

What has actually been built?

Three screens: **Overview**, **Telemetry**, and **Communication & Events**. The interface supports viewing and filtering. It has no add/edit/delete forms, live satellite connection, radio encoder, or trained AI model.

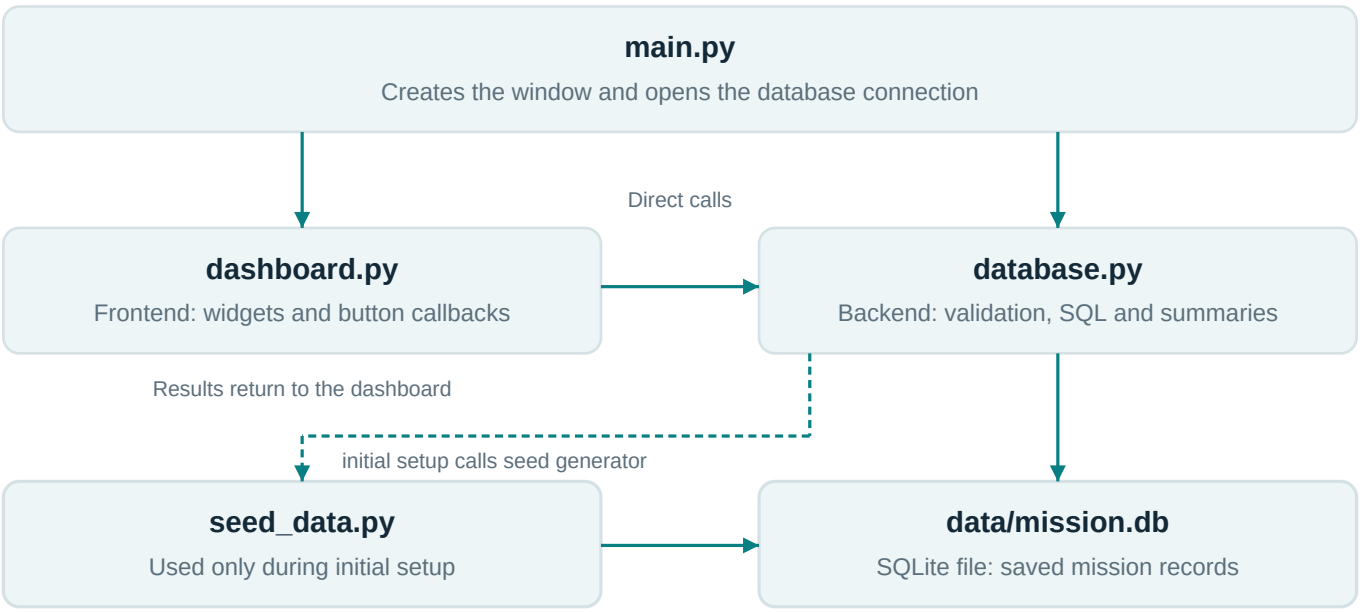
How to use this guide tonight

First read pages 2-4 to understand the connection between files. Use pages 5-8 for the database and Python concepts. Practise page 9, then answer page 10 aloud without looking.

Project context: KJS-SRS-01, SomaiyaSat & SomaiyaPod. The original mission is much larger; this application implements a small ground-station data-display component.

02 / How frontend and backend connect

The frontend and backend are parts of the **same Python program**. They communicate through function calls and share one SQLite connection. There is no web server, HTTP request, REST API, or internet dependency.



Startup, step by step

1. Running main.py creates the Tkinter root window and temporarily hides it.

2. open_database() creates the data folder, opens SQLite, and checks the schema.

3. If the telemetry table did not already exist, setup calls seed_database() to insert the sample records. Table creation and inserts are in one transaction.

4. main.py passes the connection into Dashboard(root, connection). The class stores it as self.connection and builds all three screen frames.

5. Overview loads first. The window is shown, and root.mainloop() waits for clicks, selections, and resize events.

6. When the application exits, the finally block closes the database connection.

Say this in the viva
The frontend calls database functions directly. Those functions return rows or summary values, and the frontend updates its widgets. Both parts run locally in one process.

03 / What each file does

File	Role and main responsibility
main.py	Entry point: prepares the window and connection, creates Dashboard, starts the event loop, and closes the connection on exit.
dashboard.py	Frontend: the Dashboard class builds the screens, handles user actions, applies the theme, and draws the chart.
database.py	Backend: opens SQLite, defines the three tables, checks date input, and returns filtered rows and summaries.
seed_data.py	Builds repeatable sample data and inserts it using executemany(). It receives the existing connection.
data/mission.db	Generated SQLite file containing saved records. It is data, not Python source code.
tests/test_database.py	Nine tests that use temporary databases, so checking the code does not change the demo records.
README.md / .gitignore	README explains the architecture. .gitignore keeps generated databases and Python cache files out of Git.

The important link in main.py

```
connection = open_database()
Dashboard(root, connection)
root.mainloop()
```

The same connection returned by open_database() is passed to the interface. Importing the files makes their functions and class available; the connection carries access to the database.

The backend function map

Function in database.py	What it returns or does
open_database()	Returns the initialized SQLite connection.
get_summary(connection)	Returns latest telemetry, pending count, success rate, and latest event.
get_telemetry(connection, start, end)	Returns readings matching the optional date range, newest first.
get_communications(connection, mode, status)	Returns filtered records with a calculated priority column.
get_events(connection)	Returns deployment events in chronological order.

04 / The interface and a button click

Screen	What to explain
Overview	Summary values come from get_summary(). Battery history comes from get_telemetry(), reversed into oldest-first order for the chart.
Telemetry	Date Entry widgets collect input. Apply fetches matching rows; Clear removes both dates and fetches all readings.
Communication & Events	Readonly Combobox widgets select mode/status. Selecting a row displays its notes. Communication filters do not filter the event table.

Example: show readings for 16 January 2026

1. The user opens Telemetry, enters 2026-01-16 in both date fields, and clicks Apply.

2. The button calls refresh_telemetry(). It reads the two StringVar values with .get().

3. The method calls get_telemetry(self.connection, start, end). The backend parses the dates and checks their order.

4. SQL selects rows from 00:00:00 through 23:59:59 on that date and returns a list of sqlite3.Row objects.

5. The frontend removes the old visible table items, inserts the returned rows, and updates the record count. The original sample database shows **8 readings**.

```
rows = get_telemetry(
    self.connection,
    self.start_date.get(),
    self.end_date.get()
)
```

A common misunderstanding
clear_table() removes rows from the Treeview widget only. It does not execute SQL DELETE and does not erase saved records.

Other methods to recognize

build_... methods create widgets. **show_page()** hides the other frames and refreshes the chosen one. **refresh_...** methods query and repopulate screens. **select_communication()** and **select_event()** look up notes by the selected record ID. **set_style()** controls appearance; **make_table()** supplies reusable tables and scrollbars.

05 / Inside the SQLite database

A table is like a spreadsheet: columns describe fields, and each row is one record. **INTEGER** stores whole numbers, **REAL** stores decimal numbers, and **TEXT** stores strings. Every table has an id INTEGER PRIMARY KEY, a unique row identifier.

telemetry - one health reading per row

Column	Type	Meaning / rule
id	INTEGER	Primary key
recorded_at	TEXT	Timestamp in UTC
battery_pct	REAL	Battery percentage from 0 to 100
temperature_c	REAL	Temperature in degrees Celsius
link_quality_pct	REAL	Invented link score from 0 to 100
operating_mode	TEXT	Normal or Safe

communication_records - one sample communication item per row

Column	Type	Meaning / rule
id	INTEGER	Primary key
created_at	TEXT	Creation timestamp in UTC
mode	TEXT	TT&C, SSTV, Codec2, or M17
size_kb	REAL	Positive sample size in KB
status	TEXT	Pending, Sent, or Failed
notes	TEXT	Extra explanation; defaults to an empty string

mission_events - one deployment or initialization event per row

Column	Type	Meaning / rule
id	INTEGER	Primary key
occurred_at	TEXT	Event timestamp in UTC
component	TEXT	SomaiyaPod or SomaiyaSat
event_name	TEXT	For example, Separation confirmed
outcome	TEXT	Result description; seed events use Complete
notes	TEXT	Extra explanation; defaults to an empty string

All non-ID fields are NOT NULL. CHECK constraints enforce the listed numeric ranges and allowed categories. The three tables have **no foreign keys**; the demo covers one satellite/deployer pair. There is no stored priority column: the query derives it from mode.

06 / SQL and persistence, made simple

1. SELECT retrieves rows

```
SELECT * FROM telemetry
WHERE recorded_at >= ? AND recorded_at <= ?
ORDER BY recorded_at DESC, id DESC;
```

This shows the date-filter query in simplified form. The actual function adds a condition only when that date is supplied. SELECT * means all columns; WHERE filters rows; DESC means newest first. The ID breaks ties when timestamps match.

The ? markers receive separate parameter values, such as 2026-01-16 00:00:00. Values are treated as data rather than executable SQL. The code does not paste the user's text into the query.

2. GROUP BY builds the status counts

```
SELECT status, COUNT(*)
FROM communication_records
GROUP BY status;
```

This returns one count for each stored status. Python converts the result into a dictionary, reads Sent/Failed/Pending totals, and calculates the success percentage.

3. CASE derives communication priority

```
CASE mode
    WHEN 'TT&C' THEN 1
    WHEN 'SSTV' THEN 2
    ELSE 3
END AS priority
```

This expression produces an extra column in the query result. It does not change the table. Records are ordered by priority, then created_at, then id. The program displays this ordering; it does not schedule real transmissions.

What makes the data persist?

sqlite3.connect() opens a file on disk. Startup uses CREATE TABLE IF NOT EXISTS, and seed_database() uses INSERT with executemany() for batches of rows. A transaction groups setup operations: successful completion commits them; an error rolls them back. The connection is explicitly closed by main.py on exit.

The code checks for the telemetry table before deciding to seed. An existing database is reused, even if its tables contain no rows. Pressing Refresh or changing screens only reads records; neither action reseeds the database.

Remember the difference
A connection is the open link to SQLite. A cursor holds a query result. fetchone() reads one row; fetchall() reads all result rows. sqlite3.Row allows access by column name, for example row["battery_pct"].

07 / Synthetic data and calculations

How seed_data.py works

random.Random(42) creates a local random generator with a fixed seed. A fresh database therefore receives repeatable data. datetime gives the starting time, and timedelta adds the intervals.

Dataset	Generation rule
200 telemetry readings	Begin at 2026-01-15 08:00:00 UTC; add five minutes per row. Battery changes gradually and is limited to 10-100%. Temperature and link score vary.
80 communication records	Cycle through four modes at 12-minute intervals. The last 16 are Pending. Among the first 64, every seventh index is Failed; the others are Sent.
Six mission events	A fixed sequence: preparation, release command, separation, initialization start, initialization complete, first housekeeping received.

Operational records start after initialization. Failed-transfer notes mention weak links, but communication outcomes are assigned by the sample-generation rule; they are not calculated from matching telemetry readings.

The numbers you can explain in the demo

Success rate: the original seed has 54 Sent, 10 Failed, and 16 Pending records. So 54 / (54 + 10) x 100 = **84.375%**, displayed as **84.4%**. Pending items are not attempts. If there are no attempts, the result is N/A, avoiding division by zero.

Latest health: the final sample reading is 2026-01-16 00:35:00 UTC: 32.1% battery, 27.7 degrees Celsius, a 96.3% demo link score, and Normal mode. These are reproducible examples, not real observations.

Safe mode: the generator labels a reading Safe when battery is below 20%; otherwise it uses Normal. This label and the chart threshold are classroom assumptions, not hardware commands.

How the battery chart is drawn

The frontend gets telemetry in newest-first order, reverses it, and parses its timestamps. Elapsed time determines each horizontal position; battery percentage determines vertical position. Canvas draws grid lines, a filled area, a line, the latest point, and the dashed 20% threshold.

A resize event redraws the chart. The chart uses all stored readings and is independent of the date filter on the Telemetry screen. It needs no plotting library.

08 / Python concepts used in the code

Concept	Where it appears and why
Class and self	Dashboard groups the window, connection, widgets, and callback methods. self refers to that particular Dashboard object.
Functions and imports	Modules import only the pieces they need. Database functions keep SQL separate from widget code.
Lists and tuples	The generator builds lists of row tuples. Queries return lists of rows, which loops insert into the tables.
Dictionaries	self.pages maps screen names to frames. Other dictionaries map record IDs to rows so selected notes can be found.
StringVar	A Tkinter variable stores filter or label text. .get() reads it and .set() changes it; connected widgets reflect the change.
Callbacks	A button stores a function to call later. command=self.refresh_telemetry passes the method without running it immediately.
Events and bind()	<<TreeviewSelect>> runs when a row is selected. <Configure> runs when the chart widget changes size.
Exceptions	try/except handles invalid dates and database errors. finally closes the connection after the event loop finishes.
Context managers	with connection groups database work for commit or rollback. It does not itself close the SQLite connection.
Path and datetime	Path locates the database beside the source files. strptime parses date text; strftime formats timestamps.

Two small lines worth understanding

```
command=lambda page=name: self.show_page(page)
```

The lambda creates a callback for each navigation button. page=name captures that loop iteration's screen name, so all buttons do not end up opening the final screen.

```
if __name__ == "__main__":
    main()
```

This starts the app when main.py is run directly. Importing the module does not automatically launch the window.

Layout: pack() arranges sections; grid() arranges card rows and table/scrollbar cells. They are used in different parent containers. The program uses no background threads, abstract classes, or decorators.

09 / Testing and a short demonstration

Run commands from the project folder

```
python3 main.py
python3 -m unittest discover -s tests -v
```

The application needs Python with working Tk support. python3 -m tkinter checks that installation. It uses only the standard library; no pip dependencies are required to run it.

What has been checked?

All **nine database tests passed** when preparing this guide. Each test creates a temporary database in setUp() and closes/removes it in tearDown(). The tests check saved data after reopening, repeatable generation, whole-day filters, invalid dates, mode/status filters, priority order, summary math, latest timestamps, constraints, and empty tables.

Earlier programmatic GUI checks passed for navigation, filters, notes, empty states, and the minimum window layout. These checks are different from a full manual review. Run the application on the presentation computer before the viva.

Situation	Current behavior
Invalid date / reversed range	A message box explains the error. Existing displayed rows remain.
No matching records	The table is empty and a message explains that no records match.
No transmission attempts	Success rate displays N/A.
Database problem	Startup or query handlers show a database error message.
Restart	Saved rows are reused; the initial sample data is not duplicated.

A three-minute demo

1. Open Overview. Explain the synthetic-data label, latest health reading, and battery chart.

2. Open Telemetry. Set both dates to 2026-01-16, click Apply, and show the eight sample readings. Click Clear to restore all 200.

3. Open Communication & Events. Choose SSTV and Pending, click Apply, and show four sample records. Select one to display its notes.

4. Select a deployment event and explain its note. Return to Overview and explain why pending items are excluded from the 84.4% success rate.

5. Explain that the records live in a SQLite file and remain after the app closes.

Counts above describe the original seed data. The interface currently has no add/edit/delete controls. Possible extensions include those forms, CSV export, or real telemetry ingestion; none is implemented.

10 / Viva questions and short answers

Why Python, Tkinter, and SQLite?

They cover application logic, a desktop interface, and persistent storage with standard-library modules. This suits a small offline lab project.

How are the frontend and backend linked?

main.py passes the SQLite connection to Dashboard. Button callbacks call functions in database.py, receive results, and update the widgets.

Why separate the files?

Each file has one main responsibility. We can test SQL and calculations without opening the GUI, and change the interface without rewriting the data generator.

Why a database instead of just a Python list?

Lists in memory disappear when the program exits. SQLite keeps records on disk and supports filtering, sorting, counts, and constraints.

What is a primary key? Are there foreign keys?

A primary key uniquely identifies a row. Each table has an integer ID. There are no foreign keys in this single-satellite demo.

Why use SQL placeholders?

Placeholders keep filter values separate from SQL instructions. User input is treated as data instead of being pasted into the executable query.

Does the application use AI or real satellite data?

No. It uses synthetic data and a fixed priority rule inspired by the mission. It neither controls hardware nor runs a trained model.

Is it a complete CRUD application?

No. Startup creates and inserts sample records, and the dashboard reads and filters them. The user interface has no create, update, or delete forms.

What does mainloop() do? Are we using threads?

It processes GUI events until the window closes. The application runs its small local queries in the same thread; no worker threads are used.

How do dates and Refresh work?

Dates are validated and compared against consistently formatted UTC timestamps. Refresh queries the saved data again; it does not fetch live data or generate new samples.

What are the main limitations?

Synthetic data, one satellite/deployer pair, no record-editing interface, and no live communications. The link score and battery threshold are demo assumptions.

Final revision: Explain the project in 30 seconds; trace one Apply click; name the three tables; explain the success-rate formula; state the limits honestly. Understand the flow rather than memorizing every line.

Prepared from main.py, dashboard.py, database.py, seed_data.py, tests/test_database.py, and README.md. Mission context: supplied KJS-SRS-01 use-case document. This guide describes the current code, not the full satellite mission.